1 · C++ from zero (just what the lab needs)

No C assumed. Only the pieces the 12 experiments actually use. New pieces (pointers, 2-D arrays, struct, new/delete) are taught in the experiment that first needs them.

Run a program → Program skeleton → Variables + cin/cout → if / else → Loops → Arrays → Functions → Reading errors

1.1 Running a program

A C++ program is a text file ending in **.cpp**. A **compiler** (g++) turns it into an **.exe** you can run. If the text breaks the language rules, the compiler refuses and prints **errors** with line numbers. Nothing runs until every error is fixed.

Where	How to compile + run
Code::Blocks	File → New → Empty file → save as `exp1.cpp` → **F9** (Build and run)
Dev-C++	File → New → Source File → save as `.cpp` → **F11** (Compile & Run)
VS Code	Open the folder, then in the terminal (Ctrl+`): `g++ exp1.cpp -o exp1` then `.\exp1`
This laptop	Any new terminal: `g++ -static exp1.cpp -o exp1` then `.\exp1`

SAVE AS .CPP, NOT .C

A `.c` file is compiled as C, and `cout`, `cin`, `new` and `bool &` all fail. On this laptop, always add `-static`: another program on the PATH carries an older C++ library, and without `-static` your `.exe` may exit silently.

1.2 The skeleton every experiment starts with

GUESS FIRST

Which line would you guess makes the program *start*, and what do you think `return 0;` reports, and to whom?

P01_HELLO.CPP

```cpp
#include <iostream>          // lets us use cout and cin
using namespace std;         // so we can write cout instead of std::cout

int main()                   // the program starts running here
{
    cout << "Hello, lab!" << endl;    // print, then go to a new line
    return 0;                         // 0 = "finished normally"
}
```

SCREEN (REAL RUN)

```
Hello, lab!
```

- `#include <iostream>` brings in **cout** (print) and **cin** (read). Every experiment has it.
- `using namespace std;` lets you write `cout` instead of `std::cout`. Every experiment has it.
- `int main() { ... }` is where running starts. Statements run **top to bottom**.
- Every statement ends with `;`. Blocks of statements live inside `{ }`.
- `endl` (or `"\n"` inside a string) moves to a new line.
- `return 0;` tells the operating system "finished normally".
- `//` starts a comment. The compiler ignores the rest of the line.

1.3 Variables, input, arithmetic, decisions

A **variable** is a named box that holds one value of a fixed **type**:

Type	Holds	Example
`int`	whole numbers	`int n = 5;`
`float`	numbers with decimals	`float x = 2.5;`
`char`	one character	`char c = 'A';`
`bool`	`true` / `false`	`bool done = false;`

P02_INPUT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int a, b;                          // two whole-number boxes
    cout << "Enter two numbers: ";
    cin >> a >> b;                     // read two numbers typed by the user

    int sum = a + b;
    cout << "Sum = " << sum << endl;
    cout << "a / b = " << a / b << endl;      // int / int drops the fraction
    cout << "a % b = " << a % b << endl;      // % = remainder

    if (a > b)
        cout << "a is bigger" << endl;
    else if (a == b)                   // == compares, = stores
        cout << "equal" << endl;
    else
        cout << "b is bigger" << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter two numbers: 17 5
Sum = 22
a / b = 3
a % b = 2
a is bigger
```

THE THREE CLASSIC BEGINNER SLIPS

1. `=` **stores**, `==` **compares**. `if (a = b)` compiles but is wrong.
2. `int / int` **throws away** the fraction: 17 / 5 = 3. `%` gives the remainder: 17 % 5 = 2.
3. `cin >> a >> b;` uses `>>` (data flows *into* a). `cout << x;` uses `<<` (data flows *out to* the screen).

`if (…) {…} else if (…) {…} else {…}`: only the first true branch runs. With a single statement the `{ }` are optional; with more than one they are **required**. Exp 10's manual screenshot breaks exactly this rule.

1.4 Loops and arrays

An **array** is a row of boxes of the same type, numbered from **0**. `int arr[5]` has `arr[0]` … `arr[4]`. There is **no** `arr[5]`; using it reads or writes someone else's memory, and C++ won't stop you.

P03_LOOPS_ARRAYS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};       // indexes 0..4

    cout << "for loop: ";
    for (int i = 0; i < 5; i++)              // start; keep going while true; step
        cout << arr[i] << " ";
    cout << endl;

    cout << "backwards with while: ";
    int i = 4;
    while (i >= 0)
    {
        cout << arr[i] << " ";
        i--;
    }
    cout << endl;

    int choice;
    do                                       // body runs at least once
    {
        cout << "Enter 0 to stop: ";
        cin >> choice;
    } while (choice != 0);

    return 0;
}
```

SCREEN (REAL RUN)

```
for loop: 10 20 30 40 50
backwards with while: 50 40 30 20 10
Enter 0 to stop: 4
Enter 0 to stop: 0
```

Loop	Use it when	In the lab
`for (int i = 0; i < n; i++)`	you know how many times	walking an array (every experiment)
`while (condition)`	you stop on a condition	insertion sort's shifting, linked-list walking
`do { } while (condition);`	body must run **at least once**	menus (Exp 10–12), circular list traversal

`i++` means "add 1 to i"; `i--` means "subtract 1".

> **CHECK 1.1**
> What does this print? `for (int i = 4; i > 0; i--) cout << i;`

1.5 Functions

A **function** is a named piece of code you can call. From Exp 5 on, every experiment splits its work into functions (`inputMatrix` , `insertionSort` , `enqueue` , ...).

P04_FUNCTIONS.CPP

```cpp
#include <iostream>
using namespace std;

int square(int x)                  // takes an int, gives back an int
{
    return x * x;
}

void printArray(int arr[], int n)  // void = gives nothing back
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

void doubleAll(int arr[], int n)   // arrays are NOT copied: changes are seen by main
{
    for (int i = 0; i < n; i++)
        arr[i] = arr[i] * 2;
}

void tryToChange(int x)            // plain int IS copied: main's variable is untouched
{
    x = 99;
    cout << "inside tryToChange: " << x << endl;
}

void reallyChange(int &x)          // & = reference: x IS main's variable
{
    x = 99;
}

int main()
{
    cout << "square(7) = " << square(7) << endl;

    int a[3] = {1, 2, 3};
```

```
    doubleAll(a, 3);
    printArray(a, 3);

    int v = 5;
    tryToChange(v);
    cout << "after tryToChange: " << v << endl;
    reallyChange(v);
    cout << "after reallyChange: " << v << endl;
    return 0;
}
```

SCREEN (REAL RUN)

```
square(7) = 49
2 4 6
inside tryToChange: 99
after tryToChange: 5
after reallyChange: 99
```

Read the result carefully. It explains three things you'll meet again:

- `int square(int x)`: the first `int` is what the function *gives back* via `return`; `void` means it gives nothing back.
- **An array parameter is not copied.** The function works on the caller's array, so `doubleAll` changed `a`. That's why `insertionSort(arr, n)` sorts `main`'s array.
- **A plain `int` parameter is a copy**, so `tryToChange` changed only its own copy. Add `&` (`int &x`) and the parameter *is* the caller's variable. Exp 8's `swapElements(int &a, int &b)` needs this.

> **ORDER RULE**
> A function must appear **above** the place that calls it. Write helper functions first and `main()` last, as every experiment in the manual does.

1.6 Reading compiler errors (exam-day survival)

The compiler reports `file:LINE:COLUMN: error: message`. **Fix the first error first**: one missing `;` can produce ten errors below it. Here is the manual's own Exp 12 listing, compiled:

G++ SAYS

```
m12_listing.cpp:62:17: error: missing terminating " character
   62 |     {   cout << "\n1. Insert
      |                 ^
m12_listing.cpp:13:6: error: variable or field 'insert' declared void
   13 | void insert(intvalue)
      |      ^~~~~~
m12_listing.cpp:13:13: error: 'intvalue' was not declared in this scope
   13 | void insert(intvalue)
      |             ^~~~~~~~
m12_listing.cpp:46:15: error: unused variable 'temp' [-Werror=unused-variable]
   46 |     {   Node* temp = head;
      |               ^~~~
m12_listing.cpp:55:16: error: 'temp' was not declared in this scope; did you mean 'tm'?
   55 |     }   delete temp;
      |                ^~~~
m12_listing.cpp:62:14: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'std::ostream' {aka 'std::basic_ostream<char>'})
   62 |     {   cout << "\n1. Insert
      |         ~~~~ ^~
```

Message	Usual cause
`expected ';' before …`	missing `;` on the line **above**
`'x' was not declared in this scope`	typo in a name, or a variable used outside the `{ }` it was made in
`missing terminating " character`	a string without its closing `"`
`expected '}' at end of input`	a `{` was never closed
`'else' without a previous 'if'`	a statement squeezed between `}` and `else`
`ISO C++ forbids variable length array`	`int arr[n]` with `n` read at run time; use `int arr[100]`

Answers

Check 1.1: `4321`. It starts at 4, prints while i > 0, and stops once i becomes 0.